

Problem Set Solutions: Seminar 12

Search Trees (2)
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course: Algorithms and Data Structures I
Topic: splay trees, implicit trees, lazy propagation, sequence operations
Style: Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. Splay and implicit search trees turn ordered-tree rotations into a general sequence algebra. Once split and merge are available, insertions, deletions, range sums, range updates, reversals, and even structural transformations all reduce to isolating a segment, applying a local operation, and merging it back.

Contents

Problem 1. Uniformly choosing a key from a search tree	2
Problem 2. Inserting $1, 2, \dots, n$ into a splay tree	2
Problem 3. Using a splay tree as a stack	3
Problem 4. A bamboo after inserts and why erase must splay	3
Problem 5. Split and merge in a splay tree	3
Problem 6. Implicit search tree for insert, erase, and range sum	4
Problem 7. Implicit tree with range add and range reverse	4
Problem 8. Swapper	5
Problem 9. Repeatedly duplicating the chosen middle element	5
Problem 10. Binary sequence: longest equal block, flip, reverse, sort	6
Problem 11. Moving two pointers and reversing the active interval in $O(1)$	7

Conventions. All logarithmic bounds for splay trees are amortized; for randomized treaps they are expected. Positions are one-based unless explicitly stated otherwise.

Problem 1. Uniformly choosing a key from a search tree

Solution.

Idea. A search tree gives fast ordered operations, but uniform random choice is an array operation. Keep both representations synchronized.

Algorithm. Maintain a dynamic array V of pointers to all tree vertices. In every vertex store its current index in V . To choose a key, draw a uniformly random integer $i \in \{0, \dots, |V| - 1\}$ and return $V[i]$.

When inserting a new key, insert it into the tree and append its vertex pointer to V . When deleting a vertex v , let $i = v.\text{idx}$. Swap $V[i]$ with the last array element, update the moved vertex's index, and pop the last element. Then delete v from the tree in the usual way.

Correctness. At every moment V contains every key exactly once. A uniform random index in V therefore induces the uniform distribution on the keys. The swap-delete operation preserves the invariant because it removes exactly the deleted vertex and keeps the moved vertex's back pointer correct.

Running time. The random-key operation is $O(1)$. Insert and erase pay their ordinary tree cost plus $O(1)$ array maintenance.

Problem 2. Inserting $1, 2, \dots, n$ into a splay tree

Solution.

Idea. The newly inserted key is always larger than all previous keys, hence it is inserted immediately to the right of the current maximum. Since splay trees move the accessed vertex to the root, the last inserted key becomes the root.

Algorithm. After inserting 1, the tree has one vertex. Suppose after inserting $i - 1$ the root is $i - 1$ and its left spine contains $i - 2, i - 3, \dots, 1$. Insert i as the right child of the root and splay it. This is one zig rotation, so i becomes the root, with the old tree as its left subtree.

Correctness. By induction, after the i -th insertion the root is i , all smaller keys are in its left subtree, and the tree is a left bamboo $i, i - 1, \dots, 1$. This is a valid BST because keys decrease along left edges.

Running time. Each insertion after the first performs one descent step and one rotation, hence the whole sequence costs $O(n)$ actual time.

Problem 3. Using a splay tree as a stack

Solution.

Idea. A stack only touches the most recently inserted key. In the splay representation from the previous problem, this key is already at the root.

Algorithm. Represent the stack element at position i by key i . A push inserts the new maximum key. As above, it is inserted as the right child of the root and splayed by one rotation. A pop deletes the maximum key, which is the root; its left subtree becomes the new tree.

Correctness. The maximum key is exactly the top of the stack. Push creates a new maximum and moves it to the root. Pop removes the root, hence removes the most recently inserted element. The remaining keys are precisely the previous stack contents.

Running time. Both operations take $O(1)$ actual time in this special access pattern, so any sequence of stack operations takes linear time.

Problem 4. A bamboo after inserts and why erase must splay

Solution.

Answer. The sequence

insert 1, insert 2, ..., insert n

produces the left bamboo with root n and path $n, n-1, \dots, 1$.

If erase does not splay the accessed vertex, then the sequence

erase 1, erase 2, ..., erase n

costs

$$n + (n-1) + \dots + 1 = \Omega(n^2),$$

because key 1 is at depth $n-1$, then key 2 is at depth $n-2$, and so on.

Correctness. The first statement follows from Problem 2. For the second, if deletion only removes the found vertex and performs no splay, deleting the current leaf at the bottom of the bamboo does not rebalance or move the next leaf upward enough to avoid the same pathology.

In a correct splay-tree erase, the searched vertex is splayed to the root before structural deletion. The rotations are exactly what the amortized analysis pays for; they destroy the long untouched path and prevent the quadratic sequence.

Running time. Without splaying on erase the displayed sequence is $\Omega(n^2)$. With standard splay erase, each operation is amortized $O(\log n)$, and many stack-like special sequences are even linear.

Problem 5. Split and merge in a splay tree

Solution.

Algorithm. Assume all keys of T_1 are smaller than all keys of T_2 . To compute $\text{merge}(T_1, T_2)$, splay the maximum vertex of T_1 to the root. This root has no right child. Attach T_2 as its right child.

To split a tree by key x , search for x or for the last vertex reached by the search and splay it to the root. Then detach the appropriate child so that the left tree contains keys $< x$ and the right tree contains keys $\geq x$. If one wants split by rank, first find the vertex of the required rank using subtree sizes and splay it.

Correctness. After splaying the maximum of T_1 , all keys of T_1 remain in its left side and there is no right child; attaching T_2 preserves the BST order because every key of T_2 is larger. For split, splaying the boundary key places all smaller keys on one side and all larger keys on the other; detaching the boundary edge therefore gives the required partition.

Running time. Each operation is dominated by one splay, so its amortized time is $O(\log n)$.

Problem 6. Implicit search tree for insert, erase, and range sum

Solution.

Idea. The key is not stored explicitly. The position of a vertex is determined by the size of its left subtree. This gives an implicit sequence tree.

Algorithm. Use a splay tree or randomized treap. Each vertex stores its value, subtree size, and subtree sum. Implement

$$\text{split}(T, k) = (A, B)$$

where A contains the first k elements and B contains the rest.

To insert x at position pos , split by pos , create a singleton vertex x , and merge the three pieces. To erase position pos , split before it and after it, discard the singleton middle piece, and merge the two outer pieces. To query the sum on $[l, r]$, split into A, B, C where B is exactly the segment, output $\text{sum}(B)$, and merge back.

Correctness. Subtree sizes make split by position exact. Merge preserves the inorder order of the sequence. Since subtree sums are recomputed from children after every rotation or merge, the stored sum of the isolated middle tree is exactly the requested range sum.

Running time. Each split or merge costs $O(\log n)$ amortized/expected time, and every operation uses $O(1)$ such calls.

Problem 7. Implicit tree with range add and range reverse

Solution.

Idea. The previous implicit tree becomes a sequence data structure once we add lazy tags.

Algorithm. In each vertex store value, size, sum, and two lazy tags: *add* and *rev*. Applying $add = x$ to a subtree increases its root value by x , its sum by $x \cdot \text{size}$, and accumulates the lazy tag. Applying *rev* swaps the two children and toggles the reversal flag. Before descending, push the tags to the children.

Insert, erase, and range sum are as in Problem 6. To add x on $[l, r]$, split out the segment, apply the add tag to it, and merge back. To reverse $[l, r]$, split out the segment, toggle its reverse tag, and merge back.

Correctness. Lazy add is correct because every element of the subtree is shifted by the same amount, so the subtree sum changes by x times its size. Lazy reverse is correct because the inorder traversal of a tree with swapped children is reversed. Since every operation isolates exactly the affected segment before applying a tag, no outside element is changed.

Running time. Every operation uses a constant number of split/merge calls and $O(1)$ tag operations, hence takes $O(\log n)$.

Problem 8. Swapper

Solution.

Idea. Swapping $(a_l, a_{l+1}), (a_{l+2}, a_{l+3}), \dots$ only exchanges elements between odd and even positions. Store the two parity subsequences separately.

Algorithm. Build two implicit trees: T_0 stores elements currently in odd positions of the array, T_1 stores elements currently in even positions. Each node stores subtree sum. A range $[l, r]$ corresponds to one interval in each parity tree. Because $r - l + 1$ is even, the number of odd-position and even-position elements inside it is the same.

For a sum query, split the appropriate intervals from both trees, add their stored sums, and merge back. For a swap query, split out the two corresponding parity intervals and exchange them between T_0 and T_1 ; then merge back.

Correctness. Inside an even-length range, each adjacent pair consists of one odd-position element and one even-position element. Swapping every pair therefore exchanges exactly the two parity subsequences restricted to that range, while preserving the order inside each subsequence. The two-tree operation performs precisely this exchange.

Running time. Each query performs $O(1)$ splits and merges on implicit trees, hence costs $O(\log n)$. Building the trees takes $O(n)$.

Problem 9. Repeatedly duplicating the chosen middle element

Solution.

Idea. The array length stays fixed: the operation removes both ends and inserts two additional copies of a_i around the old a_i . This is a sequence operation.

Algorithm. Use an implicit treap/splay tree. For a query with index i , first read $x = a_i$ by splitting or descending by size. Delete the first and last elements. In the remaining sequence, the old a_i is at position $i - 1$ if $1 < i < n$. Insert two new singleton vertices with value x immediately after that position, so three consecutive copies of x appear.

For boundary cases use the analogous definitions: if $i = 1$, keep the left value and insert copies at the beginning after deleting elements from the far end; if $i = n$, do the symmetric operation at the end. Finally output the inorder traversal.

Correctness. For $1 < i < n$, deleting the first and last elements transforms

$$(a_1, \dots, a_n)$$

into

$$(a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1}).$$

Inserting two copies of a_i adjacent to the surviving copy gives exactly

$$(a_2, \dots, a_{i-1}, a_i, a_i, a_i, a_{i+1}, \dots, a_{n-1}).$$

The boundary definitions are the same limiting operation when one side is absent.

Running time. Each query is $O(\log n)$ with an implicit balanced tree. The final traversal is $O(n)$.

Problem 10. Binary sequence: longest equal block, flip, reverse, sort

Solution.

Idea. A binary segment can be summarized by a constant amount of information: length, number of ones, boundary values, longest equal prefix/suffix, and best equal block. Lazy tags handle complementing, reversing, and assigning all bits.

Algorithm. Use an implicit treap. For each subtree store

$$\text{size}, \quad \text{cnt}_1, \quad \text{pref}_0, \text{pref}_1, \quad \text{suff}_0, \text{suff}_1, \quad \text{best}_0, \text{best}_1,$$

where best_b is the longest all- b block in the subtree. These fields are recomputed from the two children and the root bit in $O(1)$.

Lazy tags:

- flip swaps all 0-metadata with 1-metadata and changes the root bit;
- reverse swaps children and swaps prefix with suffix;
- $\text{assign}(b)$ makes the whole subtree equal to b .

For query (a), split out $[l, r]$ and return $\max(\text{best}_0, \text{best}_1)$. For (b) and (c), split out the segment and apply flip or reverse. For (d), split out $[l, r]$, let $c = \text{cnt}_1$. Split the segment after size $-c$ positions, assign the first part to 0 and the second to 1, then merge.

Correctness. The stored summaries are exactly the dynamic-programming information needed to combine two adjacent binary strings. Flip and reverse preserve correctness because their effect on summaries is purely symmetric. Sorting a binary segment means replacing it by all zeros followed by all ones; the count of ones uniquely determines the two lengths.

Running time. Each operation uses $O(1)$ splits/merges and $O(1)$ lazy updates, so the time is $O(\log n)$.

Problem 11. Moving two pointers and reversing the active interval in $O(1)$

Solution.

Answer. Yes.

Algorithm. Maintain three deques:

$$A = a_1, \dots, a_{l-1}, \quad B = a_l, \dots, a_r, \quad C = a_{r+1}, \dots, a_n.$$

Also maintain one Boolean flag saying whether B is logically reversed. Operations on the logical front/back of B are implemented by using the physical front/back according to this flag.

To reverse $[l, r]$, only toggle the flag. To move l left, move the last element of A to the logical front of B . To move l right, move the logical front of B to the back of A . To move r right, move the first element of C to the logical back of B . To move r left, move the logical back of B to the front of C . Queries for a_l and a_r read the logical front and logical back of B .

Correctness. The invariant is that the logical concatenation $A + B + C$ is always the current array and B is exactly the active interval. Every pointer move transfers exactly one boundary element between adjacent deques, so the invariant is preserved. Reversal of the active interval changes only the orientation of B , so toggling the flag is sufficient.

Running time. Every operation performs a constant number of deque operations. Thus each query is worst-case $O(1)$.